

DATATALKSCLUB

MLZOOMCAMP 2025

Week 6 notes

Carlos Leal León

November 3, 2025

General Index

1	Credit Risk Scoring	2
1.1	Data Cleaning	2
1.2	Decision Tree	2
1.3	Structure	2
1.4	Decision Tree Algorithm	3
1.5	Decision Tree Parameter Tuning	4
1.6	Ensemble learning	5
1.6.1	Random Forests	5
1.6.2	Bootstrapping	5
1.6.3	Parameter Tuning	5
1.7	Gradient Boosting	6
1.8	XGBoost	6
1.8.1	XGBoost tuning	6
2	Homework 06 Notes	7
2.1	Prepare Data	7
2.2	Train a Decision Tree Regressor	7
2.3	Train a Random Forest Regressor	7
2.4	Use different values for estimator	7
2.5	Use different values for max_depth and estimator	7
2.6	Feature importance from tree based model	8
2.7	Train an XGBoost model	8

1 Credit Risk Scoring

To introduce Decision Trees you have to use the credit scoring data `CreditScoring.csv` [1]. The goal here is to determine whether a bank should lend a loan to a client or not. A zero means the client is likely to payback the loan, a one means the opposite.

1.1 Data Cleaning

When you start exploring the dataset it is mentioned that it was previously cleaned but using R. Taking a look at the values of some categorical variables lets you identify that categorical values start with a value of 1 and continue from there this is relevant as what you want is to use the categorical value of each categorical variable.

Now when you take a look at the numerical features you realize that there are features with max values of 99999999 which from the R cleaning process indicate an unknown value so it is recommended to replace this values with `np.nan`.

The outcome variable has an extra output which is not of interest you simply remove it from the dataset.

Now is time to partition the dataset into train, validation and test using a 60, 20, 20 proportion, remember to remove the outcome variable from each partition and to have it available as a numeric type in a separate variable.

1.2 Decision Tree

A Decision Tree starts with a single node and a condition whether this condition is True or False determines which way the Decision Tree moves and each subsequent decision has a different condition. It may look like a bunch of if-else conditionals depending on how many features are taken into account to build the depth of the tree.

1.3 Structure

Is composed of nodes (conditions) and branches (values for a condition).

Start with a root node which has other nodes connected to it, in the last level there are terminal nodes called leafs.

The depth is the number of levels of a tree or the levels from the root node to the farthest node.

To create a Decision Tree you use `DecisionTreeClassifier` [2] in this case the categorical variables were encoded using `DictVectorizer` as in previous homework, then you complete the training process and use validation data to see how well the model performs.

To evaluate the performance the `roc_auc_score` is used on the validation and training data and the results let you see what the model is overfitting which means that the model memorizes the data but is not able to generalize when new data is available. This happens because the tree is growing too much in depth and is being specific about the data that is using to learn.

To see the names of the features taken into account in the decision process you can use `sklearn.tree` and `import_export_text`. Example:

```
print(export_text(dec_tree, feature_names=dict_vect.get_feature_names()))
```

1.4 Decision Tree Algorithm

The Decision Tree classification process in the most basic form takes into account a Classification criteria [3].

I said in the most basic form because each feature value is compared against all the available feature values that are referenced as thresholds, if the feature value is above the threshold it is assigned to one of the classes of the outcome and so if it is below.

Once values have been assigned to each of the outcome classes you can calculate the impurity of each of the thresholds and determine which threshold gives the less impurity and select the decision criteria that best generalizes that feature.

There are different techniques as listed in the reference for Classification criteria but the goal is the same prevent overfitting.

It is important to remember that this process does not only consider feature values but also the depth of the tree, how many splits at each node and other attributes of trees which affect the way it grows and the criteria evaluated in each node.

1.5 Decision Tree Parameter Tuning

Decision Trees have different parameters that can be tuned. But two of them have their importance highlighted. Based on how they help in controlling overfitting, bias and variance.

- Control Overfitting
 - `max_depth`: Limits tree complexity (growing too deep) when it grows excessively it memorizes training data. Affecting the ability to recognize unseen data.
 - `min_samples_leaf`: Ensures that leaf nodes have a sufficient number of samples. Reducing the chance to create nodes that are specific to the training data.
- Model Bias and Variance
 - Low bias, high variance
 - * An increase in `max_depth` and decrease in `min_samples_leaf`. Leads to a complex model.
 - High bias, low variance
 - * An decrease in `max_depth` and increase in `min_samples_leaf`. Leads to a simpler model.

The nature of the data and objective help you decide how much bias and variance can change. An optimal balance between them can serve as a good starting point.

1.6 Ensemble learning

Multiple models referred as weak learners are combined to solve a particular computational problem. Yields superior predictive performance compared to only using a single model.

One good example on ensemble learning are Random Forests.

1.6.1 Random Forests

A Random Forest is composed of a number of Decision Trees. So each model is a Decision Tree and from each model their predictions are aggregated to identify the most repeated result. Random Forests only select a subset of features from the dataset to make predictions.

Important aspects of Random Forests:

- Training: Each Decision Tree is potentially trained on a bootstrapped sample of the original data. This introduces randomness at the row level. Each Decision Tree is trained independently.
- During tree construction, at each node only a random subset of features is considered to make a split. This helps to prevent overfitting and promote generalization for unseen data. The feature randomness helps decorrelate Decision Trees.

1.6.2 Bootstrapping

Resampling technique where subsets of data are created using sampling with replacement. Making some datapoints appear multiple times in a single subset while others may be excluded. This way each Decision Tree is trained on a distinct sample contributing to the diversity and robustness of the model.

1.6.3 Parameter Tuning

It is a technique where model parameters are used with different combinations of values to optimize the performance in this case of a machine learning model such as Random Forests. The improvement in model performance comes with an increase in computational cost, increases when more data is processed.

1.7 Gradient Boosting

In Gradient Boosting each model takes the prediction errors made by the previous model and from there it tries to improve. After a number of iterations all the predictions get combined to make the final prediction.

1.8 XGBoost

The XGBoost library implements the gradient boosting technique. Install it using *pip install xgboost*. In order to train and evaluate a model you have to wrap your train and validation data in a data structure called *DMatrix* which is optimized to train XGBoost models faster.

1.8.1 XGBoost tuning

The highlighted parameters are:

- `eta`: Used to prevent overfitting by regularizing the weights of new features in each boosting step. Defaults to 0.3.
- `max_depth`: Increasing this parameter increases model complexity thus increase overfitting similar to trees. Default 6.
- `min_child_weight`: Minimum number of samples in leaf node. Default 1.

To try different model parameters it is advised to try different parameters using a loop to train different models (basic approach).

In general XGBoost models perform better on tabular data but they tend to overfit because of the high number of hyperparameters so you have to be careful and pay attention to detail when you are tuning them.

2 Homework 06 Notes

For this homework you use the `car_fuel_efficiency` dataset from homework 02.

2.1 Prepare Data

First of all you have to Prepare the dataset by:

- Handling missing values (fill with zeros).
- Partition data in train, validation, test with 60, 20, 20 distribution with `random_state 1`.
- Encode categorical features using `DictVectorizer` with `sparse set to True`. This to turn dataframes into matrices.

I reused code from previous homework just adjusted for the target feature.

2.2 Train a Decision Tree Regressor

For this I used `DecisionTreeRegressor` [4] for the model. And to visualize the split made by the tree I used `export_text` from `sklearn.tree` [5].

The model only uses the `max_depth` parameter to visualize how the split is made and what is the feature used for it.

2.3 Train a Random Forest Regressor

For this I used `RandomForestRegressor` [6] for the model.

The model only uses:

- `n_estimators`: Set to 10, which is the number of trees.
- `random_state`: Set to 1.
- `n_jobs` (optional): This uses the number of cpu cores for concurrent processes.

For the RMSE I reused code from homework 02 which is just a function receiving actual `y` values and predicted `y` values.

2.4 Use different values for estimator

Here you just have to place the training code for the Random Forest Regressor inside of a loop to change the `n_estimators` value and replace it in the model parameter in each run.

With each pass you can calculate the RMSE of the used estimator and save it in a list or dictionary so you can access it later. You have to round it to three decimals.

2.5 Use different values for max_depth and estimator

Here you have to add an extra change by using the `max_depth` parameter. To do this placing the training code with the changes to the estimator inside of another loop can help, inside that loop now you change the values of `max_depth` for the model with each pass.

Here also consider increasing `n_jobs` to a suitable value or if you have the computational power set to `-1`.

2.6 Feature importance from tree based model

After training the RandomForestRegressor you have to:

- Get feature names: It is required to identify which feature has the most importance. Get it from the DictVectorizer `get_feature_names_out()`.
- Get feature importance: You access this from the `model.feature_importances_` attribute.
- Zip names and importances: Use `zip` to pair each feature name with its corresponding importance.
- Sort (optional): You can sort the result to identify most important feature faster.

2.7 Train an XGBoost model

To train an XGBoost [7] model consider:

- Get feature names: Feature names are used when you create DMatrix from your data splits.
- Train split DMatrix: Use the train and `y_train` splits and turn them into DMatrix.
- Validation split DMatrix: Use the validation and `y_validation` splits and turn them into DMatrix.
- Create a watchlist: Is used to monitor performance of model during training. I have a note below about this.
- Xgb params: It is important that you set the `verbosity` and `eval_metrics` parameters or you are going to face errors. Keep in mind that the objective parameter changes depending on the type of problem you are trying to solve.
- Capture output: To capture the output when the model is trained use `%%capture` output in the cell containing the code to train the model. Then access `stdout` attribute and process each line as you wish. You can create a dataframe after processing so you can plot the `eval_metrics`.

Note: I realized that my train and validation splits were showing the same result during training. So I started to track what was the cause. I found out that when I made the splits the indexes of each split were reset referencing the train split and the validation and test splits were referencing data from the train split. I made the adjustments and then ran all cells again. This was thanks to the watchlist output.

References

- [1] gastonstat (github). "CreditScoring" [Online], at: credit-scoring-data [Consulted on October 2025].
- [2] scikit learn. "DecisionTreeClassifier" [Online], at: decision-tree-classifier [Consulted on October 2025].
- [3] scikit learn. "Decision Trees" [Online], at: decision-tree-classification-criteria [Consulted on October 2025].
- [4] scikit learn. "Decision Tree Regressor" [Online], at: decision-tree-regressor [Consulted on October 2025].
- [5] scikit learn. "Export Text" [Online], at: export-text [Consulted on October 2025].
- [6] scikit learn. "Random Forest Regressor" [Online], at: random-forest-regressor [Consulted on October 2025].
- [7] XGBoost (DMLC). "XGBoost" [Online], at: xg-boost [Consulted on November 2025].